

05 Tree-Based Models

Markéta Tranberg, mlap@viauc.dk

March 2026

Table of Contents

1 Decision trees

2 Random forests

3 Gradient-boosted trees

4 Code example

Classification and regression

- Classification
 - Mode/probability
 - Gini impurity
 - Discrete output based on mode
- Regression
 - Mean
 - MSE/MAE
 - Discrete output based on mean

Representation of a decision tree

Graph representation

Binary Directed Acyclic Graph (DAG)

- **Nodes:** binary question on an attribute
- **Edges:** outcomes of the question
- **Leaves:** final prediction/class

Feature Space Representation

Recursive partitioning

- Non-overlapping regions covering the feature space
- Each leaf node in the graph is one unique isolated region

Representation of a decision tree (salary/age)

Graph representation

**Feature Space
Representation**

CART (=Classification And Regression Trees) algorithm

- 1 For each feature, find the threshold that gives the *best* binary division
- 2 Divide according to the best division
- 3 Create subtrees
- 4 Repeat until:
 - All leafs have the same category
 - No more splits can be found
 - The *hyperparameters* say to stop

Time complexity

- k : number of features
- n : number of samples
- d : depth of tree

Time complexity of a single split:

Time complexity for the whole tree:

Gini Impurity

Decides what the **best split** is based on the probability of a specific element being **incorrectly** classified if it was randomly labelled.

$$G_i = 1 - \sum_j \left(\frac{n_{i,j}}{n_i} \right)^2$$

- n_i : number of samples in i th node
- $n_{i,j}$: number of samples in i th node of j th category
- G_i : Gini impurity of i th node

$G_i = 0 \approx$ pure node

Example

X_1	X_2	Color
+	-6	Magenta
+	-7	Magenta
-	-2	Green
-	4	Green
+	-2	Magenta
+	2	Yellow
-	0	Green
+	11	Yellow
+	2	Yellow

Example

X_1	X_2	Color
+	-6	Magenta
+	-7	Magenta
-	-2	Green
-	4	Green
+	-2	Magenta
+	2	Yellow
-	0	Green
+	11	Yellow
+	2	Yellow

Pros and cons of trees

Pros:

- Fast
- Easily interpretable
- Visualizable
- Invariant to scaling
- Handles both numerical and categorical data types
- Captures non-linear relationships (complex regions)
- Natural identification of feature importance

Cons:

- Prone to overfitting
- High variance (dependence on data)
- Unable to predict smooth trend in regression

Hyperparameters

Implementing pre-pruning ($\approx$ early-stopping)

- `max_depth`: max questions per branch
- `min_samples_split`: number of samples in a node necessary for a split
- `min_samples_leaf`: max samples in the final split
- `max_leaf_nodes`: max number of leafs in the whole tree
- `criterion`: split criteria (default Gini, MSE possible)

Ensembles

= more powerful (and complex) model by combination of multiple ML models

Random forest

Collection of slightly different trees, each overfit differently

Gradient-boosted decision trees

Sequence of trees, each tries to correct the mistakes of the previous one

Table of Contents

1 Decision trees

2 Random forests

3 Gradient-boosted trees

4 Code example

Random forest

Each tree is deterministic.

Random forest:

- row randomness (bootstrapping),
- column randomness (feature subsetting).

Many trees are built in parallel.

Bootstrapping

= bagging

- Random sampling of rows with replacement
- Some are chosen multiple times
- OOB (= Out-of-Bag)
Samples were not chosen
- used for scoring

<u>X</u>	<u>X</u>	<u>X</u>
X_1		
X_2		
X_3		
X_4		
X_5		
X_6		
X_7		
X_8		
X_9		
X_{10}		

Feature subsetting

Each tree only looks at a random subset of features.

The algorithm is forced to find patterns in the columns it is given: de-correlation.

Hyperparameter: `max_features`: how many features the subset has.

- `max_feature = 1`: very different trees (low correlation)
- `max_feature = n_features`: no randomness
- `max_feature =  $\sqrt{n\_features}$` : rule of thumb

Predictions from Random trees

Classification:

- Majority vote across trees (older, simpler version)
- Average of predicted probabilities of each tree
(`scikit-learn`)

Regression: mean of means

Pros and cons of random forests

Pros:

- Very powerful
- Reduced overfitting compared to trees
- High accuracy
- Built-in validation (OOB error)

Cons:

- Slower
- Difficult to interpret (black box)
- Dependent on random splits
- Computationally expensive
- Regression: cannot extrapolate (predictions outside of range)

Table of Contents

- 1 Decision trees
- 2 Random forests
- 3 Gradient-boosted trees**
- 4 Code example

Gradient-boosted tree

Sequence of trees, each tries to correct mistakes of the previous one - sequential improvement.

- 1 Initial prediction
- 2 Calculate error (residuals)
- 3 Train a tree on the residuals
- 4 Update the prediction (overall error becomes smaller)
- 5 Repeat

Gradient boosting

Hyperparameters

- `n_estimators`: how many trees in sequence (too few underfit, too many overfit)
- `max_depth`: depth of each tree
- `learning_rate`: scales contribution of each tree ($\approx 0.1 - 0.3$ might work well)

Pros and cons of gradient-boosted decision trees

Pros:

- Very powerful
- Flexible loss function
- Feature importance (which feature reduces error)

Cons:

- Requires careful parameter tuning
- Prone to overfitting
- Computationally expensive
- Sensitive to outliers

Comparison: Tree, Forest, and Boosting

Feature	Decision Tree	Random Forest	Gradient Boosting
Logic	Single Model	Parallel (Bagging)	Sequential (Boosting)
Main Goal	Explainability	Reduce Variance	Reduce Bias
Tree Depth	Deep	Deep	Shallow (Stumps)
Overfitting	Very High	Low	Moderate (if tuned)
Tuning	None	Low	High (Learning Rate)
Speed	Fast	Slowest	Slower
Best For	Simple Rules	Noisy Data	Max Accuracy

Table of Contents

- 1 Decision trees
- 2 Random forests
- 3 Gradient-boosted trees
- 4 Code example**

Code example

Github link: https://github.com/304125/MAL1_S26/tree/master/05_Tree-Based_Models